

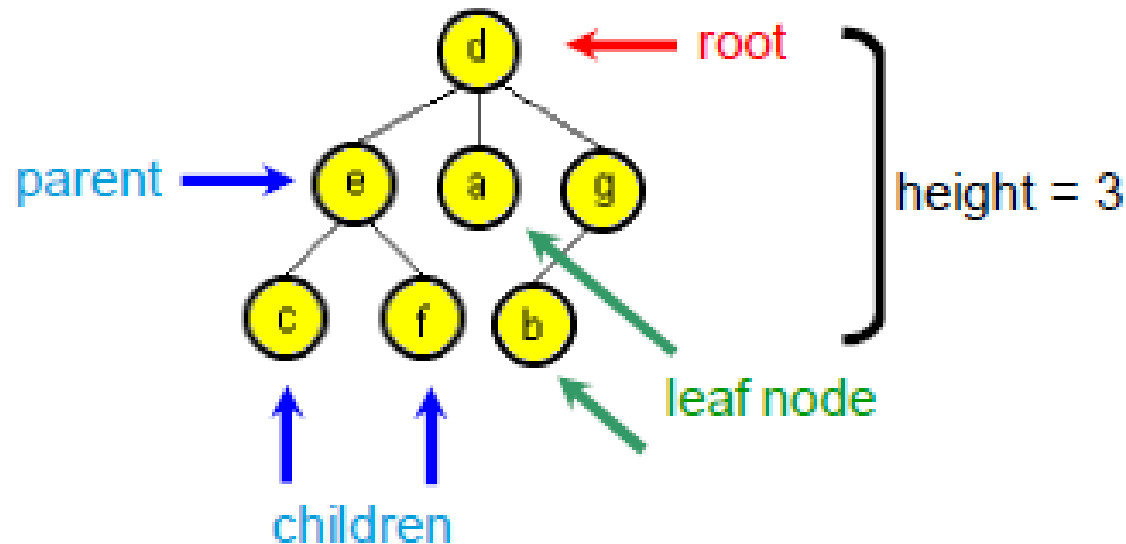


Heap Sort



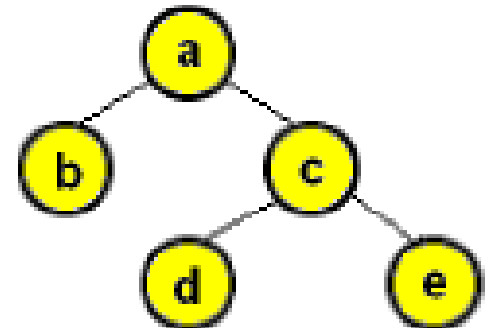
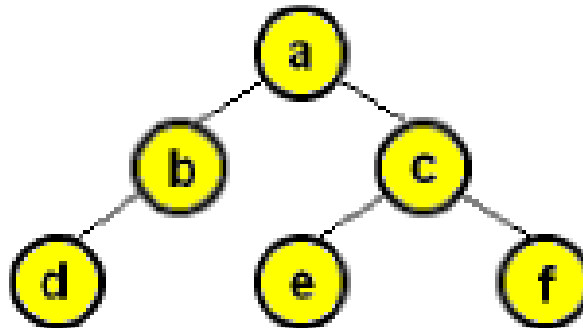
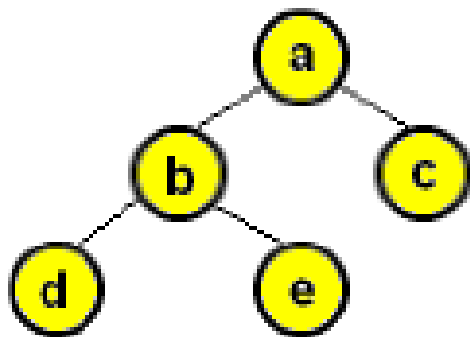
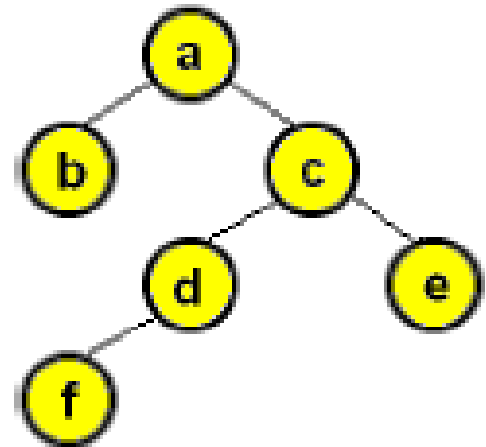
Tree

Tree ▶ suatu struktur data yang setiap elemen data (diistilahkan dengan simpul, **node**) berelasi dengan beberapa elemen data lainnya



Binary Tree

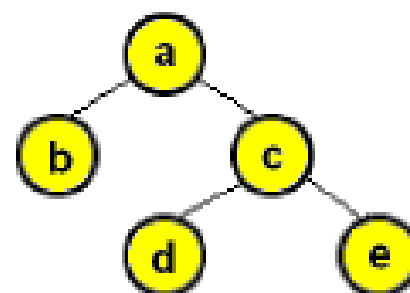
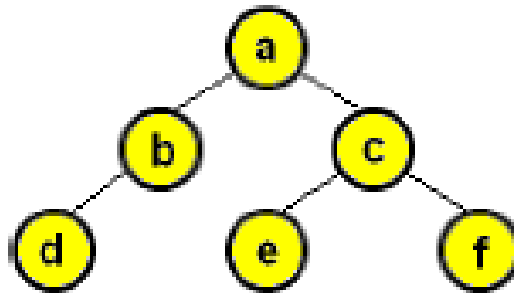
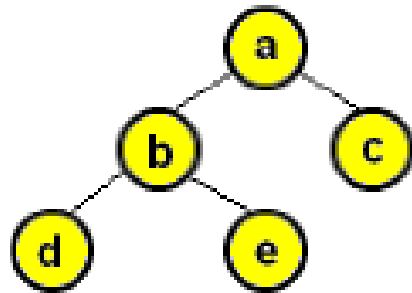
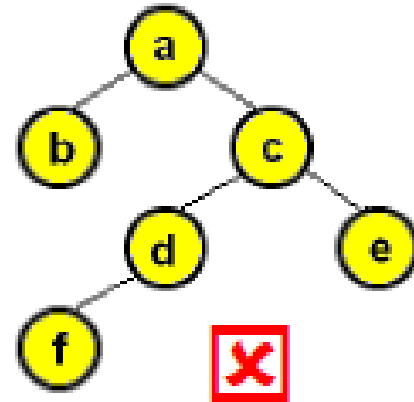
Binary Tree ▶ *tree* yang setiap simpulnya mempunyai paling banyak dua simpul anak (*children*)



(Special) Complete Binary Tree

(Special) Complete Binary Tree

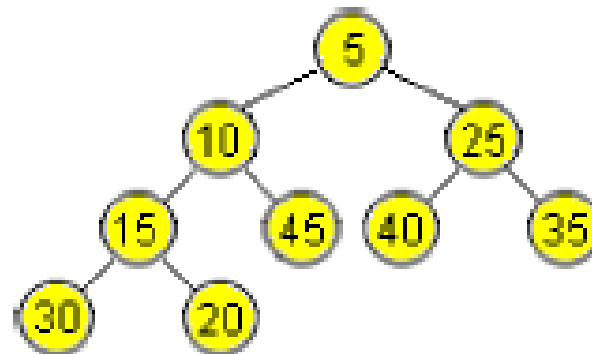
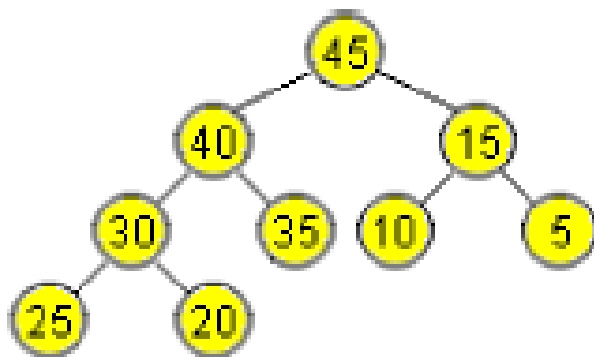
▶ sejenis *binary tree* dengan *height* minimum dan simpul pada level paling bawah tersusun mulai dari posisi kiri



Max Heap dan Min Heap

Max Heap ▶ adalah *complete binary tree* yang nilai setiap simpul induk lebih besar daripada nilai simpul anak

Min Heap ▶ adalah *complete binary tree* yang nilai setiap simpul induk lebih kecil daripada nilai simpul anak

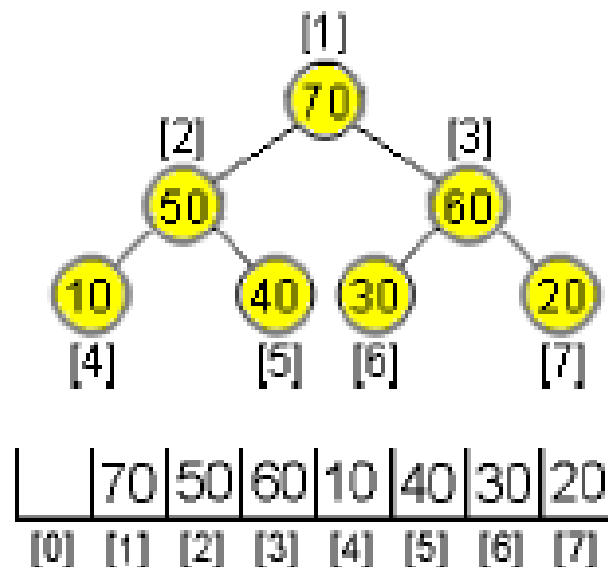


Heap

Implementasi ▶ ditampung dengan array (linier)

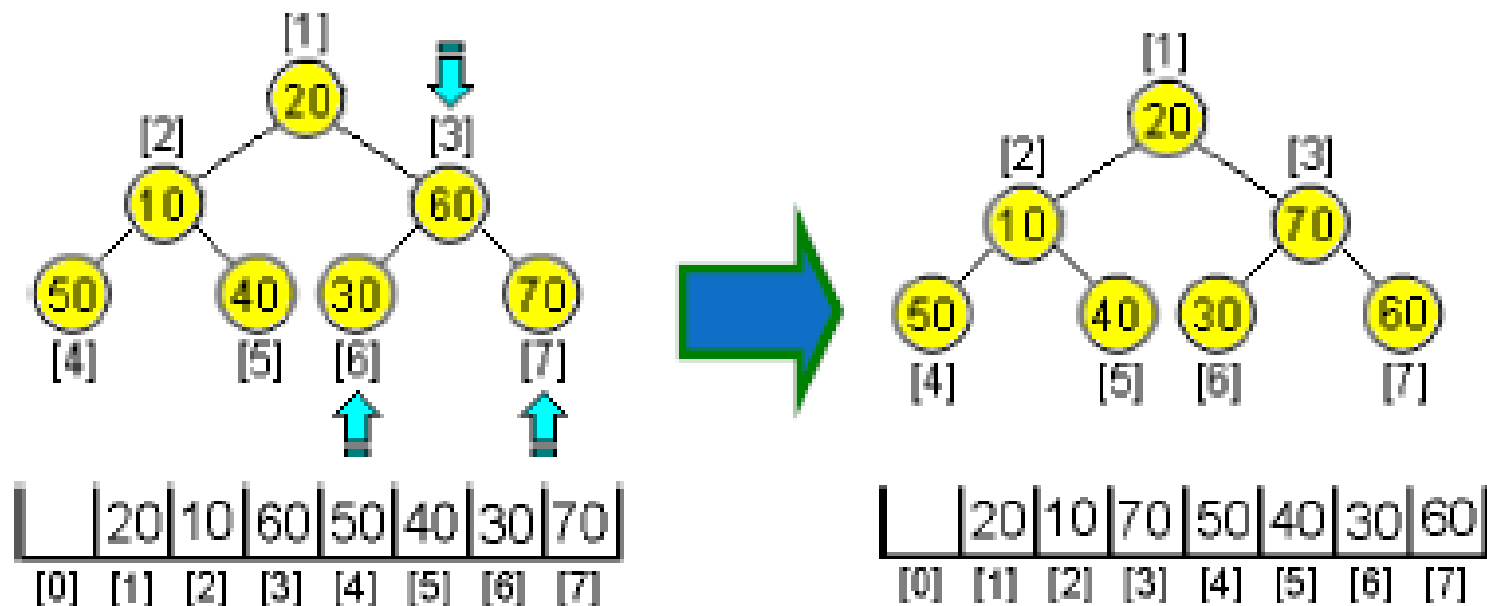
root di [0] ▶ simpul induk [i], simpul anak $[2*i+1]$ dan $[2*i+2]$

root di [1] ▶ simpul induk [i], simpul anak $[2*i]$ dan $[2*i+1]$

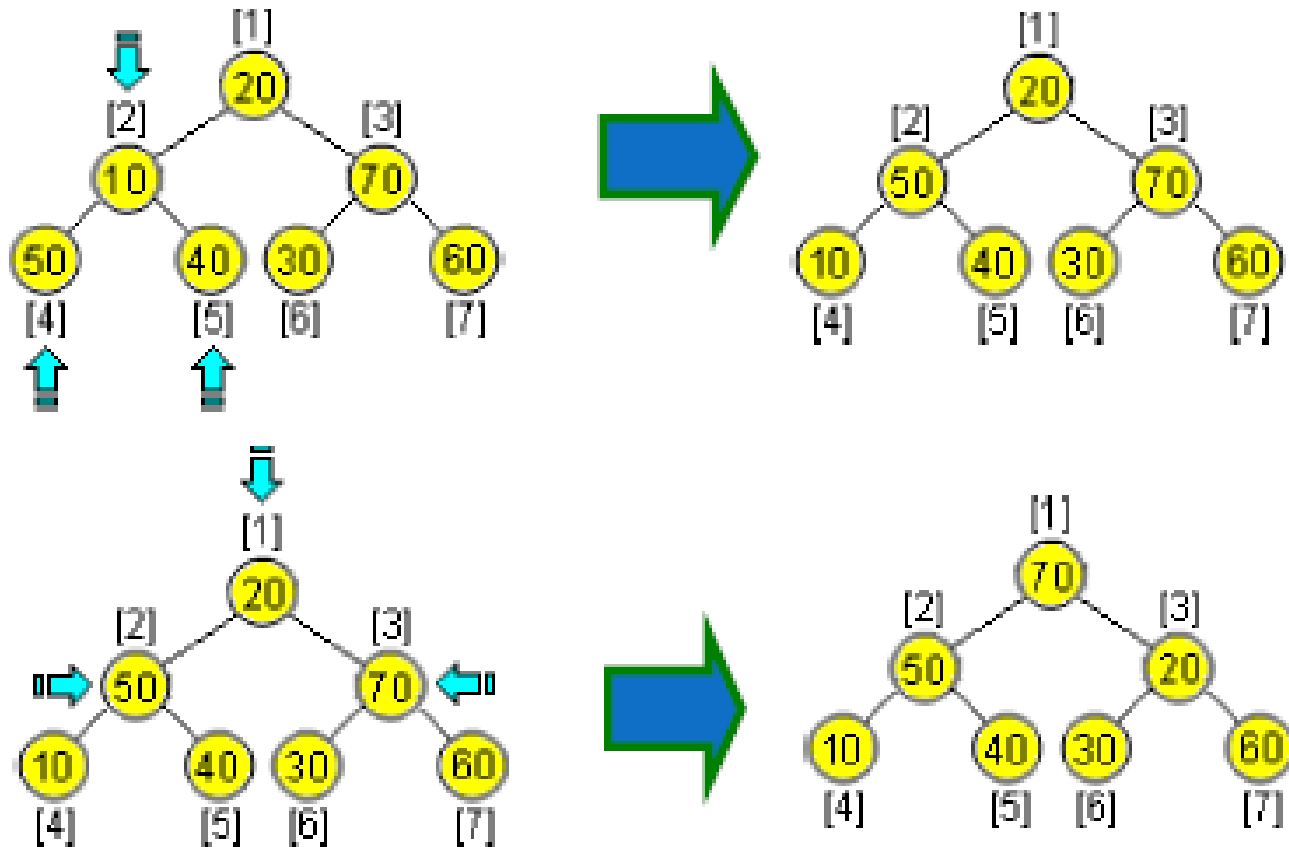


Heapify

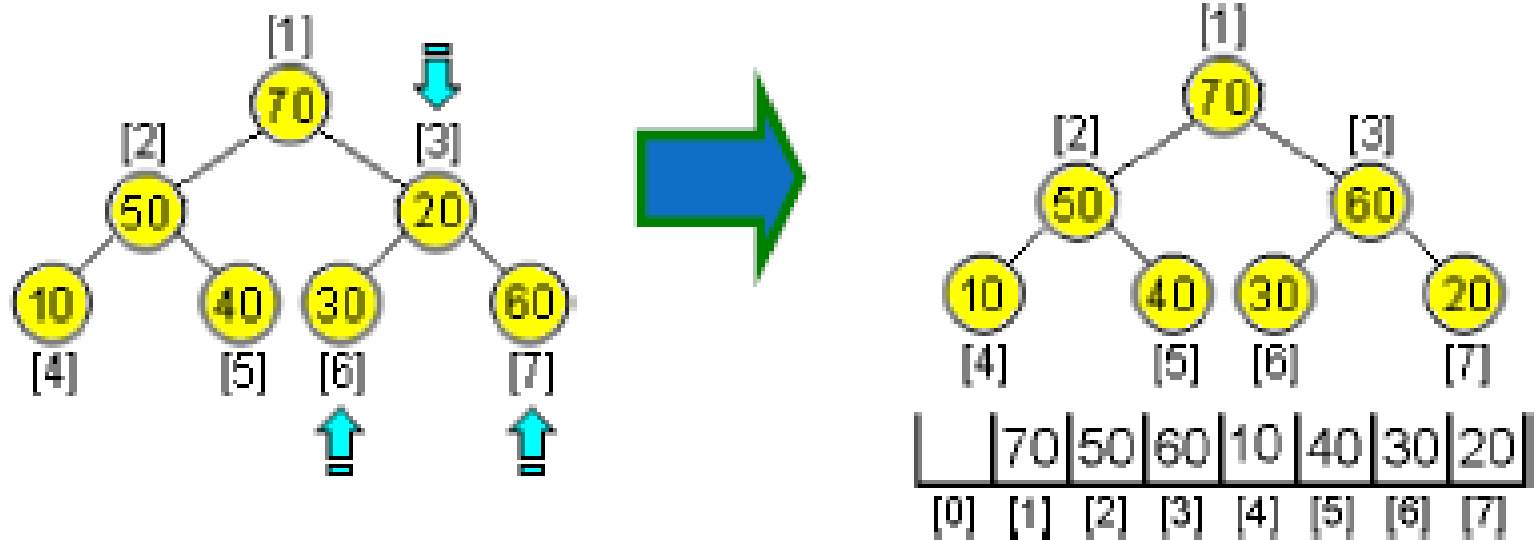
Heapify ► proses mengubah complete binary tree menjadi struktur heap. Pembetulan dilakukan terhadap simpul yang mempunyai anak (dimulai pada indeks $[n/2]$)



Heapify



Heapify



Program Heap

```
void heapify (int arr[], int i, int n){
    int idxanak, temp;

    while (i <= n/2){
        //Menentukan apakah left node atau right node yg lebih besar
        if (arr[2*i + 1] > arr[2*i]) idxanak = 2*i + 1;
        else idxanak = 2*i;

        //Jika node anak lebih besar dari induknya (arr[i]), maka tukar posisi
        if (arr[idxanak] > arr[i]){
            temp = arr[i]; //node induk disimpan sementara di tmp
            arr[i] = arr[idxanak];
            arr[idxanak] = temp;
        }else break; //Jika tidak ada pertukaran, selesai

        //Terjadi pertukaran, cek lagi subnode dibawahnya
        i = idxanak;
    }
}
```



Program Heap

```
int main() {  
  
    int data [1000];  
    int i, n, j, jdata;  
  
    scanf("%d", &jdata);  
    for (i = 1; i <= jdata; i++) scanf("%d", &data[i]);  
    for (i = jdata/2; i > 0; i--){  
        heapify (data, i, jdata);  
        for (j = 1; j <= jdata; j++){  
            printf("%d ", data[j]);  
        }  
        printf("\n");  
    }  
  
    getchar();  
    getchar();  
    return 0;  
}
```

9									
5	10	15	20	25	30	35	40	45	
5	10	15	45	25	30	35	40	20	
5	10	35	45	25	30	15	40	20	
5	45	35	40	25	30	15	10	20	
45	40	35	20	25	30	15	10	5	

Heap Sort Ascending

Heapsort secara *ascending* ▶ **max heap**.

Pada mulanya data dibenahi sehingga menjadi struktur *heap*.

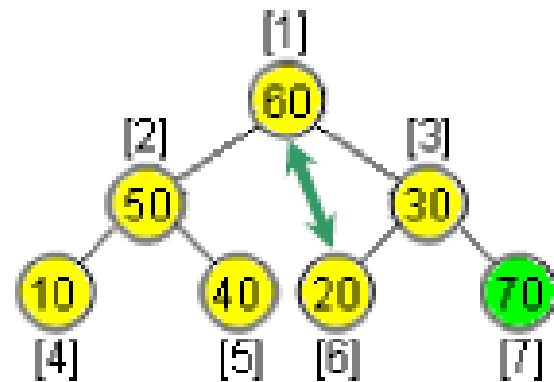
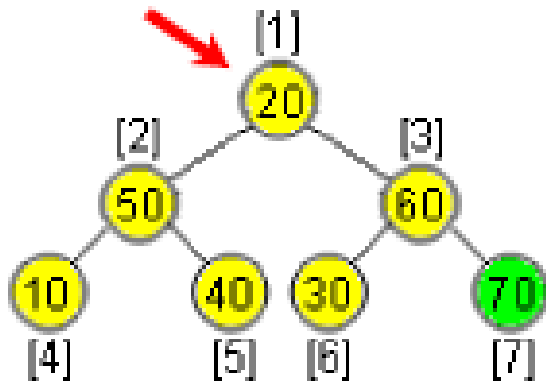
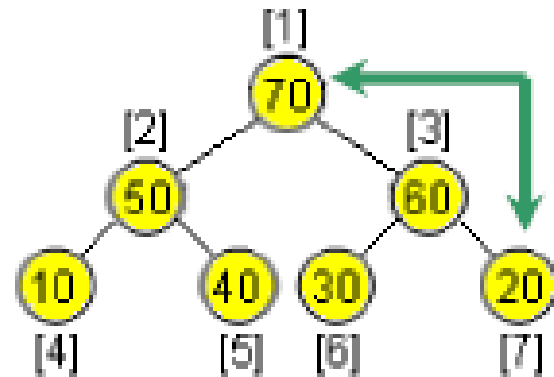
Kemudian dilakukan proses berulang:

- 1) elemen pada *root* ditukar dengan elemen terakhir,
- 2) elemen terakhir hasil pertukaran dianggap tidak lagi menjadi bagian dari *heap*,
- 3) terhadap elemen pada *root* kemudian dilakukan proses *heapify*

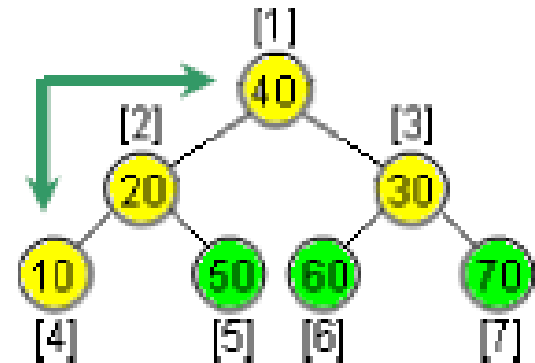
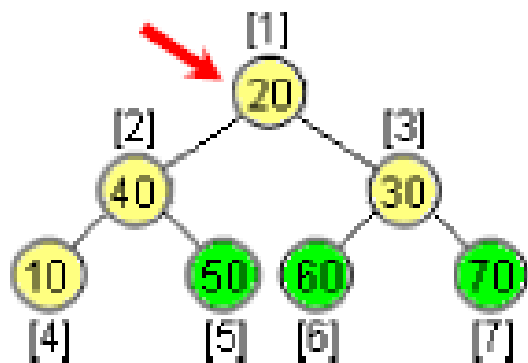
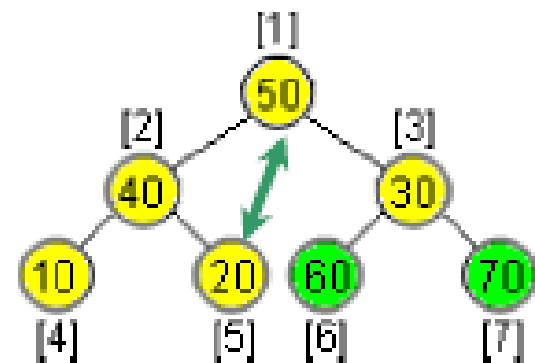
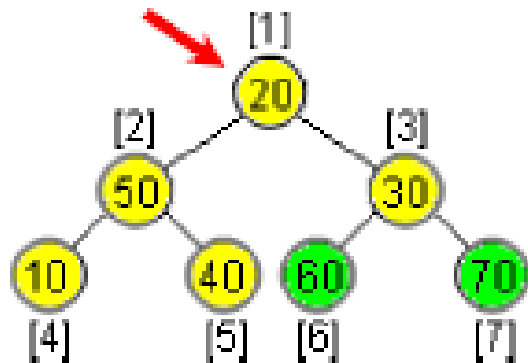


Heapsort

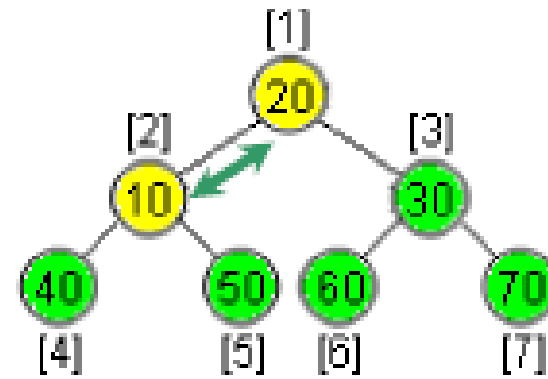
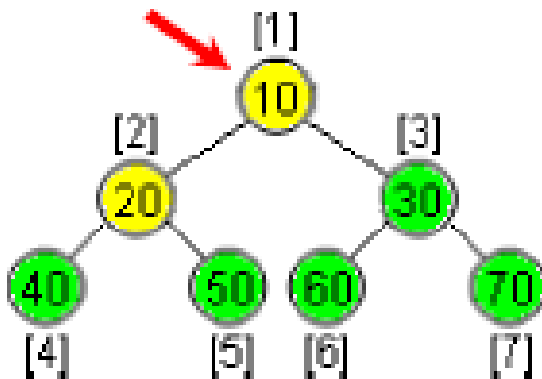
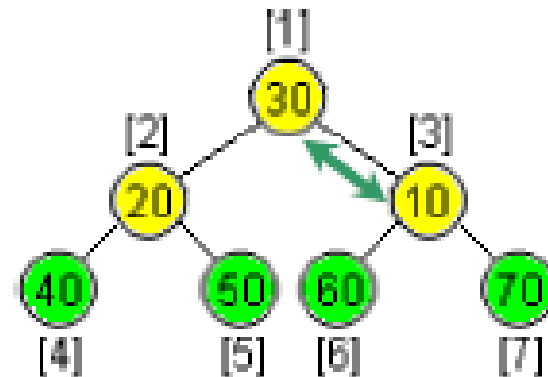
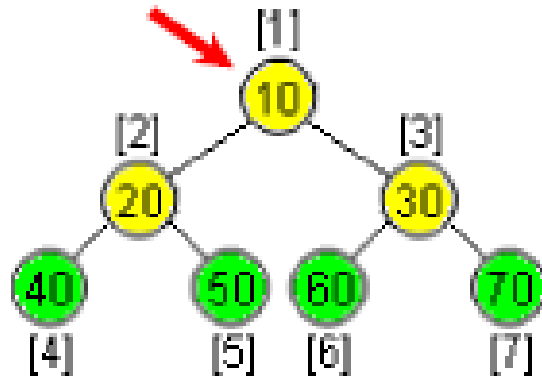
	70	50	60	10	40	30	20
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



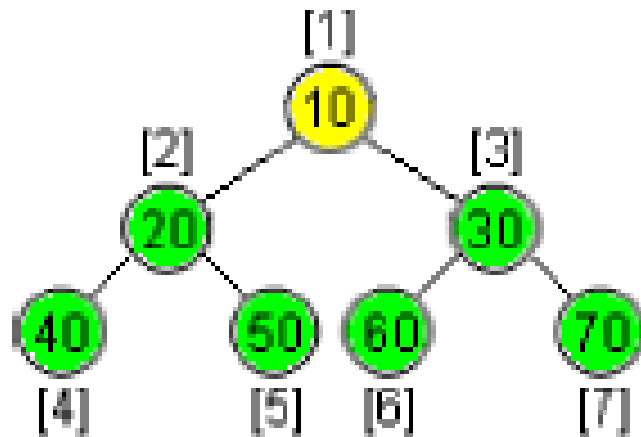
Heapsort



Heapsort



Heapsort

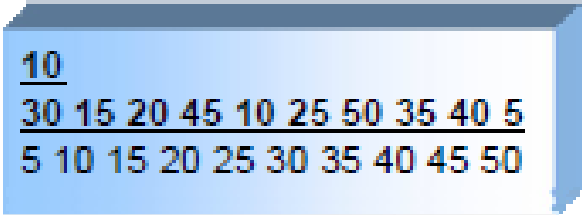


	10	20	30	40	50	60	70
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



Program Max Heap

```
# include <stdio.h>
void heapify(int arr[], int i, int n) {
    ...
}
int main() {
    int data[1000], i, jdata, temp;
    scanf("%d", &jdata);
    for (i = 1; i <= jdata; i++) scanf("%d", &data[i]);
    for (i = jdata / 2; i > 0; i--)
        heapify(data, i, jdata);
    for (i = jdata; i > 1; i--) {
        temp = data[i]; data[i] = data[1];
        data[1] = temp; heapify(data, 1, i-1);
    }
    for (i = 1; i <= jdata; i++) printf("%d ", data[i]);
    return 1;
}
```



10
30 15 20 45 10 25 50 35 40 5
5 10 15 20 25 30 35 40 45 50

Contoh Penerapan

Sebuah printer yang terhubung dalam jaringan dipakai bersama oleh sejumlah user. Printer ini dilengkapi memori yang cukup besar untuk menampung file-file yang masuk. File yang dikirim ke printer ini diberi nilai prioritas berdasarkan user. Nilai yang lebih tinggi menyatakan prioritas yang lebih tinggi. Antrian file seperti ini disebut PRIORITY QUEUE

Apa struktur data yang paling efisien untuk mengimplementasi priority queue dalam hal memilih file yang akan dicetak?



Alternatif Algoritma

Sort list:

- 1) setiap kali ada file baru yang masuk ke print queue maka seluruh data di-sort kembali ($O(n^2)$),
- 2) saat printer tersedia ambil file pada [0] untuk dicetak.

Unsorted list:

- 1) file yang masuk ke printer queue diletakkan pada posisi akhir,
- 2) saat printer tersedia pilih file dengan prioritas tertinggi ($O(n)$) untuk dicetak

Max heap:

- 1) setiap kali ada file baru yang masuk ke print queue lakukan proses heapify ($O(2 \log n)$),
- 2) saat printer tersedia ambil file pada [1] untuk dicetak,
- 3) delete elemen pada [1], tukar dengan elemen terakhir, heapify ($O(2 \log n)$).



Referensi

1. Ngoen, T.S., (2004), *Data Structures*, IBII Lecture Notes
2. Mark Allen Weiss, (1997), *Data Structures and Algorithm Analysis in C*, Addison Wesley Longman, inc.
3. Horowitz,E., Sahni,S. and Anderson-Freed,S.,(1993), *Fundamentals of Data Structures in C*, Computer Science Press,New York.



Sartaj Sahni
Univ. of Florida



Thompson S.N.
IBII Jakarta



Mark Allen Weiss
Florida Int'l Univ.



Ellis Horowitz
Univ. of Southern
California

Latihan

	20	50	40	70	10	60	30
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

- ▶ Diberikan array di atas yang merupakan representasi dari tree.
 - ▶ Lakukan simulasi **Heapfy** pada tree tersebut di atas untuk mendapatkan **Min Heap**. Min Heap adalah *complete binary tree* yang nilai setiap simpul induk lebih kecil daripada nilai simpul anak.
 - ▶ Kemudian lakukan simulasi **Heap Sort** pada Min Heap yang dihasilkan tersebut untuk mendapatkan urutan secara **descending** (menurun/mengecil).

